

```
!pip install diffusers transformers accelerate

from huggingface_hub import login
from google.colab import userdata

login('hf_gVjArVVeOiNjuWwiSnyHLExAdZyASYgGP',
add_to_git_credential=True)

import torch
from diffusers import FluxPipeline

pipe = FluxPipeline.from_pretrained(
    "black-forest-labs/FLUX.1-schnell",
    torch_dtype=torch.float32, # Chạy trên CPU cần float32
    low_cpu_mem_usage=True
).to("cpu") # Chuyển sang CPU

generator = torch.Generator(device="cuda").manual_seed(0)
prompt = "A futuristic class full of students learning AI coding in the
surreal style of Salvador Dali"

# Generate the image using the GPU
image = pipe(
    prompt,
    guidance_scale=0.0,
    num_inference_steps=4,
    max_sequence_length=256,
    generator=generator
).images[0]

image.save("surreal.png")
```

Hàm/Dòng lệnh	Chức năng
!pip install diffusers transformers accelerate	Cài đặt các thư viện diffusers, transformers và accelerate.
from huggingface_hub import login	Import hàm đăng nhập từ Hugging Face Hub.
from google.colab import userdata	Import mô-đun userdata của Google Colab (không sử dụng trong code).
login('hf_gVjArVVeOiNjuWwiSnyHLExAdZyASYgGP', add_to_git_credential=True)	Đăng nhập vào Hugging Face bằng API token và lưu thông tin đăng nhập vào Git credentials.
import torch	Import thư viện torch để sử dụng GPU và tính toán tensor.
from diffusers import FluxPipeline	Import FluxPipeline từ thư viện diffusers để sinh ảnh.
pipe = FluxPipeline.from_pretrained("black-forest-labs/FLUX.1-schnell", torch_dtype=torch.bfloat16).to("cuda")	Tải mô hình FLUX.1-schnell của black-forest-labs, sử dụng độ chính xác bfloat16 và chạy trên GPU.
generator = torch.Generator(device="cuda").manual_seed(0)	Tạo bộ sinh số ngẫu nhiên trên GPU với seed là 0 để đảm bảo kết quả tái lập.

prompt = "A futuristic class full of students learning AI coding in the surreal style of Salvador Dali"	Định nghĩa mô tả cho ảnh cần sinh.
image = pipe(prompt, guidance_scale=0.0, num_inference_steps=4, max_sequence_length=256, generator=generator).images[0]	Sinh ảnh dựa trên mô tả prompt với guidance_scale=0.0, num_inference_steps=4 và max_sequence_length=256.
image.save("surreal.png")	Lưu ảnh sinh ra với tên surreal.png.

<pre>from IPython.display import display from PIL import Image # Display the image inline display(image)</pre>	
Hàm/Dòng lệnh	Chức năng
from IPython.display import display	Import hàm display để hiển thị hình ảnh trong notebook Jupyter.
from PIL import Image	Import thư viện PIL để xử lý hình ảnh.
display(image)	Hiển thị hình ảnh được sinh ra (image) trực tiếp trong notebook.

<pre>!pip install datasets from transformers import pipeline from datasets import load_dataset import soundfile as sf import torch synthesiser = pipeline("text-to-speech", "microsoft/speecht5_tts", device='cuda') embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors", split="validation") speaker_embedding = torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0) speech = synthesiser("Hi to an artificial intelligence engineer on the way to mastery!", forward_params={"speaker_embeddings": speaker_embedding}) sf.write("speech.wav", speech["audio"], samplerate=speech["sampling_rate"]) from IPython.display import Audio # Play the generated audio Audio("speech.wav")</pre>	
Hàm/Dòng lệnh	Chức năng
!pip install datasets	Cài đặt thư viện datasets để làm việc với tập dữ liệu trên Hugging Face.
from transformers import	Nhập pipeline từ thư viện transformers để sử dụng các

pipeline	mô hình AI.
from datasets import load_dataset	Nhập load_dataset để tải tập dữ liệu từ Hugging Face.
import soundfile as sf	Nhập soundfile để xử lý và lưu trữ file âm thanh.
import torch	Nhập thư viện torch để làm việc với tensor và mô hình AI.
synthesiser = pipeline("text-to-speech", "microsoft/speecht5_tts", device='cuda')	Tạo pipeline tổng hợp giọng nói từ văn bản bằng mô hình speechT5 trên GPU.
embeddings_dataset = load_dataset("Matthijs/cmu-arctic-xvectors", split="validation")	Tải tập dữ liệu nhúng giọng nói cmu-arctic-xvectors.
speaker_embedding = torch.tensor(embeddings_dataset[7306]["xvector"]).unsqueeze(0)	Chuyển đổi nhúng giọng nói thành tensor và thêm chiều batch.
speech = synthesiser("Hi to an artificial intelligence engineer on the way to mastery!", forward_params={"speaker_embeddings": speaker_embedding})	Tổng hợp giọng nói từ văn bản với nhúng giọng nói.
sf.write("speech.wav", speech["audio"], samplerate=speech["sampling_rate"])	Lưu âm thanh tổng hợp vào file speech.wav.
from IPython.display import Audio	Nhập Audio để phát file âm thanh trong notebook.
Audio("speech.wav")	Phát file âm thanh đã tổng hợp.

```
import torch
from diffusers import StableDiffusionPipeline

model_id = "OFA-Sys/small-stable-diffusion-v0"
pipe = StableDiffusionPipeline.from_pretrained(model_id,
torch_dtype=torch.float16)
pipe = pipe.to("cuda")
pipe.enable_model_cpu_offload()

prompt = "A futuristic class full of students learning AI coding in the surreal style of Salvador Dali"
image = pipe(prompt).images[0]

from IPython.display import display
from PIL import Image

# Display the image inline
display(image)
```

Hàm/Dòng lệnh	Chức năng
import torch	Nhập thư viện torch để sử dụng PyTorch và GPU.

<code>from diffusers import StableDiffusionPipeline</code>	Nhập StableDiffusionPipeline từ thư viện diffusers để tạo ảnh bằng mô hình Stable Diffusion.
<code>model_id = "OFA-Sys/small- stable-diffusion-v0"</code>	Xác định ID của mô hình Stable Diffusion nhỏ gọn.
<code>pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_dtype=torch.float16)</code>	Tải mô hình Stable Diffusion với độ chính xác float16 để tối ưu bộ nhớ.
<code>pipe = pipe.to("cuda")</code>	Chuyển mô hình sang GPU để tăng tốc xử lý.
<code>pipe.enable_model_cpu_offload() ()</code>	Cho phép chuyển bớt phần mô hình không sử dụng về CPU để tiết kiệm RAM GPU.
<code>prompt = "A futuristic class full of students learning AI coding in the surreal style of Salvador Dali"</code>	Định nghĩa câu lệnh mô tả hình ảnh cần tạo.
<code>image = pipe(prompt).images[0]</code>	Tạo hình ảnh dựa trên prompt và lấy ảnh đầu tiên trong danh sách kết quả.
<code>from IPython.display import display</code>	Nhập display từ IPython.display để hiển thị hình ảnh trong notebook.
<code>from PIL import Image</code>	Nhập PIL.Image để xử lý hình ảnh.
<code>display(image)</code>	Hiển thị hình ảnh đã tạo trong notebook.